

基于 Triton 语言的 FlagGems 高性能 GPU 算子开发与优化

徐宇鸿

本案例记录 Kaggle Track 1 LLM Operator Development and Optimization 竞赛的工程实践，使用 OpenAI Triton 语言为 FlagGems 开源 GPU 算子库开发高性能深度学习算子。案例对象是大语言模型推理与训练中频繁调用的 20 个算子，难度分为基础逐元素运算、卷积池化与损失算子，以及线性代数分解、序列损失、空间插值、线性注意力融合三档。文中分析 Triton 的 Block 级 SPMD 编程抽象与 MLIR/LLVM 编译流水线，梳理 FlagGems 的 libentry、pointwise_dynamic 与运行时配置体系，以 `log10`、`max_pool3d`、`upsample_nearest2d`、`conv_transpose2d`、`svd`、`chunk_gated_delta_rule` 等算子为例，讨论逐点模板生成、Tile 并行索引、反向传播散射与基于 autotune 的软流水调优。交付的 20 个算子通过 2871 项功能测试且无一失败，多数算子相对 PyTorch 原生实现取得 $1.0\times$ 至 $2.8\times$ 的设备端加速，部分融合算子加速比超过 $15\times$ 。本工作的竞赛提交对应 FlagGems 仓库 Pull Request #3460，见 <https://github.com/flagos-ai/FlagGems/pull/3460>。

第一部分 理论基础与核心概念

本部分为算子开发实践建立理论基础。第 1.1 节说明 GPU 算子性能为何成为深度学习系统的关键瓶颈，第 1.2 节讨论 GPU 的硬件特征与并行优化难点，第 1.3 节介绍 Triton 语言的编程抽象与编译流水线，第 1.4 节梳理 Triton 的核心接口，第 1.5 节分析 FlagGems 算子库的架构与算子分类。

一、大模型时代的高效算子开发需求

1.1 LLM 推理与算子性能瓶颈

随着大语言模型规模从数亿参数扩展至数千亿，推理与训练的计算成本急剧上升。一次前向传播中，仅注意力机制就涉及 softmax 归一化、掩码、缩放点积、dropout 等十余种逐元素或归约算子，前馈层还包含矩阵乘法、层归一化、激活函数与残差连接。若这些算子逐一调用通用的 PyTorch eager 实现，中间张量会在 GPU 显存与寄存器之间反复搬运，带来巨大的内存带宽开销，即内存墙问题。

算子性能取决于两个核心指标。计算密度指每字节搬运完成的浮点运算数 FLOPs/Byte，Roofline 上界由峰值算力与显存带宽共同决定。逐元素算子如 `log10`、`cosh` 计算密度极低，性能完全受限于显存带宽，理想实现应使有效带宽逼近硬件峰值。归约与矩

阵列类算子如 median、卷积、SVD 存在计算与访存的混合瓶颈，需要通过分块与数据复用提升计算密度。判断算子属于何种瓶颈，是制定优化策略的前提。

1.2 GPU 计算特征与并行优化挑战

现代 GPU 如 NVIDIA Hopper/Ada 架构以 SIMT 单指令多线程方式执行程序，内存层次自上而下分为寄存器、共享内存与 L1 缓存、L2 缓存、全局显存、主机内存，容量递增而延迟递增（图 1）。高效算子应让数据驻留在靠近计算单元的存储层级，并通过合并访问减少对全局显存的访问次数。

直接用 CUDA C/C++ 编写达到上述目标的算子并不容易，开发者需要手动管理线程块划分、共享内存分配、bank conflict 规避、双缓冲流水线，以及冗长的模板代码。这套底层细节对深度学习研究者构成显著门槛。Triton 语言的出发点正是在保留接近手写 CUDA 性能的同时，把编程抽象提升到块一级。

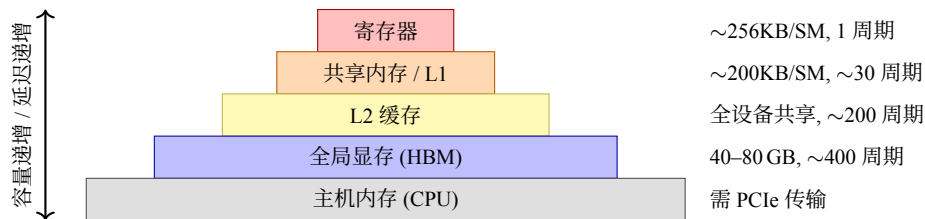


Figure 1 GPU 的内存层次：越靠近计算单元容量越小、延迟越低，算子优化的核心是让数据驻留在高层

1.3 Triton 语言：面向深度学习的 GPU DSL

Triton 由 OpenAI 提出，是一种嵌入在 Python 中的领域专用语言。程序员以数据块而非单个线程为单位书写运算，编译器自动完成线程块内的并行映射、共享内存分配、数据复用与内存访问合并。相比 CUDA，Triton 的代码量通常减少一个数量级。相比 PyTorch 的高层算子，Triton 允许把多个操作融合到一个 kernel 中，避免中间结果落盘显存。表 1 对比了三种开发方式的取舍。

Table 1 CUDA、Triton 与 PyTorch eager 三种算子开发方式的对比

维度	CUDA C/C++	Triton (DSL)	PyTorch eager
编程抽象	单线程 + 手动同步	数据块级 SPMD	整张量级
性能上限	最高（可极致调优）	接近手写 CUDA	受内存墙限制大
开发效率	低（模板冗长）	高（Python 语法）	最高
算子融合	完全可控	完全可控	不支持（需 torch.compile）
代码量	大	小	极小（调用 API）

正因如此，FlagGems 选择 Triton 作为其算子实现的唯一编程语言，并要求所有社区贡献也以纯 Triton 形式提交。

二、深入解析 Triton 编程模型

2.1 Block 级抽象与 SPMD 编程模型

Triton 的执行模型是单程序多数据 SPMD。程序员书写一份作用于一个程序实例的函数, GPU 上每个线程块运行一个程序实例。函数内部通过 `tl.arange(0, BLOCK_SIZE)` 声明编译期固定大小的块级向量, 对该向量的运算会被映射到线程块内数十至数百个线程上并行执行。

例如, 对一个长度为 N 的一维张量做逐元素 $y = x^2$, Triton 的写法是:

```
@triton.jit
def square_kernel(x_ptr, y_ptr, N, BLOCK: tl.constexpr):
    pid = tl.program_id(0) # 当前程序实例的编号
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offs < N
    x = tl.load(x_ptr + offs, mask=mask) # 从全局内存载入一个块
    y = x * x
    tl.store(y_ptr + offs, y, mask=mask) # 写回结果
```

编译器自动完成三件事, 把 BLOCK 个元素的载入合并为若干次 128 字节的对齐事务, 把乘法映射到线程束内的并行指令, 处理边界掩码以支持非整除长度。程序员无需关心线程索引、共享内存或同步原语。

2.2 Triton 编译流水线: 从 Python 到 PTX

Triton 程序的编译经过图 2 所示的多级中间表示。Python 函数在 `@triton.jit` 装饰下被解析为抽象语法树并特化为基于 MLIR dialect 的 Triton IR, 经死代码消除、循环不变量外提、块级数据流分析等优化 Pass 转化为 LLVM IR, 再由 LLVM 后端生成 NVIDIA PTX, 最终由驱动编译为本机 SASS 码。整个流水线对程序员透明, 关键在于 MLIR 提供了结构化的高层中间表示, 使分块、广播、归约等张量级操作可在编译期被全局优化, Triton 性能因而接近手写 CUDA。

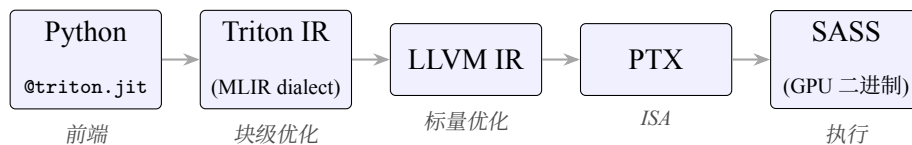


Figure 2 Triton 的多级编译流水线: Python DSL 经 MLIR/LLVM 降到 GPU 原生指令

2.3 核心编程接口: jit、autotune 与 heuristics

Triton 提供三个对算子开发至关重要的装饰器。

- `@triton.jit` 即时编译装饰器, 将 Python 函数特化为 GPU kernel。参数中标注 `tl.constexpr` 的为编译期常量, 不同取值生成不同特化版本。
- `@triton.autotune` 自动调优装饰器。对同一 kernel 预先给出多组配置, 包括块大小、`num_warps`、`num_stages`, 编译器在首次运行时逐一实测, 为每组输入形状选

择最快配置并缓存。本案例在 `max_pool3d` 反向优化中大量使用该机制。

- `@triton.heuristics` 启发式配置装饰器，根据输入张量形状在编译期推导合理常量，例如小空间维度时复用 `SAME_H` 分支，减少通用代码路径。

三者的典型组合是 `heuristics` $\rightarrow$ `autotune` $\rightarrow$ `jit`，FlagGems 几乎所有非逐点算子都遵循这一装饰顺序。

2.4 FlagGems 算子库与生态

FlagGems 由智源研究院发起，是 FlagOS 体系中的开源 GPU 算子库，定位为 NVIDIA CUB/cuDNN 的纯 Triton 替代。它实现了 PyTorch ATen 算子集的子集，用户调用 `flag_gems.enable_gems()` 后，PyTorch 的 `dispatch` 机制把对应算子路由到 Triton kernel，而非默认的 cuDNN/cuBLAS 实现。这使得 FlagGems 能在昇腾、寒武纪、海光等国产 GPU 上提供与 NVIDIA 等价的算子能力，是软硬协同生态的重要组成部分。本案例的工作是为该库贡献 20 个 Triton 算子实现及测试。

三、FlagGems 算子库架构分析

3.1 核心基础设施：libentry 与 pointwise_dynamic

FlagGems 在原生 Triton 之上构建两层基础设施，使新增算子可以高度复用，见图 3。

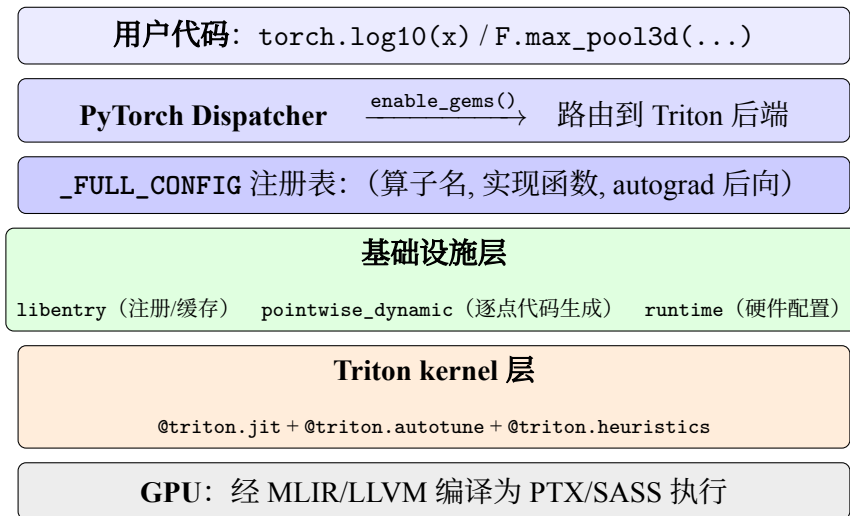


Figure 3 FlagGems 的分层架构：用户调用经 dispatcher 进入 `_FULL_CONFIG`，再由基础设施层调度到具体 Triton kernel

`libentry` 是所有顶层 kernel 的统一入口装饰器，封装了 `@triton.jit`，额外提供跨进程的 `autotune` 缓存持久化、设备上下文自动管理 `with torch_device_fn.device(...)`，以及调试日志。所有非逐点算子的 kernel 都应以 `@libentry()` 而非裸 `@triton.jit` 包装。

`pointwise_dynamic` 是代码生成器。对逐元素算子，开发者只需书写一个返回标量表达式的 `jit` 子函数，`pointwise_dynamic` 自动处理多维广播、类型提升

`promotion_methods`、输出写入与 `grid` 划分。像 `log10` 这样的算子实现只需 4 行核心代码，见代码 3。本案例的 8 个简单算子全部基于该机制。

`runtime` 模块封装硬件相关的调优配置,通过 `runtime.get_tuned_config("op_name")` 为中等与复杂算子返回适配当前设备的 `autotune` 配置，做到一份代码多硬件可调。

3.2 算子分类与计算特征

本案例的 20 个算子按计算模式归为五大类，计算特征与核心技术见表 2。这一分类既是后续章节的组织线索，也决定每个算子的优化思路。

Table 2 20 个算子的计算模式分类与核心技术

计算模式	算子	核心技术
逐元素	<code>log10</code> , <code>logaddexp</code> , <code>cosh</code> , <code>asinh</code> , <code>gcd</code> , <code>pointwise_dynamic</code> 代码生成 <code>leaky_relu</code>	
布局/索引	<code>tril</code> , <code>roll</code> , <code>scatter_reduce</code> , <code>pixel_shuffle</code>	索引重映射 + 掩码
归约	<code>median</code>	分块归约 + 二次全局合并
卷积/池化	<code>conv_transpose2d</code> , <code>avg_pool3d</code> , <code>max_pool3d</code>	Tile 并行 + <code>autotune</code>
上采样/插值	<code>upsample_nearest2d</code> , <code>grid_sample</code>	坐标映射 + 反向散射
损失	<code>smooth_l1_loss</code> , <code>ctc_loss</code>	条件分支 + 动态规划
线性代数	<code>svd</code>	Jacobi 旋转并行化
融合	<code>chunk_gated_delta_rule</code>	块状递推融合 (FLA)

3.3 LLM 推理场景中的关键算子

这些算子并非孤立存在，而是 LLM 推理路径上的高频节点。`logaddexp` 与 `log_softmax` 同源，用于注意力权重的对数域计算。`smooth_l1_loss` 与 `ctc_loss` 是训练阶段的核心损失。`grid_sample` 用于检索增强生成中的空间对齐与数据增强。`svd` 服务于模型压缩与低秩适配 LoRA。`chunk_gated_delta_rule` 是线性注意力与门控 DeltaNet 的核心融合算子，直接决定这类高效注意力变体的推理吞吐。理解算子的场景价值，有助于在性能与精度之间做出权衡。

第二部分 实践指南：20 个 Triton 算子开发与优化

本部分把第一部分的理论付诸实践。先搭建开发与测试环境，建立可复用的工程流程，再按基础逐点、中等复杂度、困难场景的顺序逐一剖析 20 个算子的实现，重点在分块策略、反向传播设计与自动调优，最后给出全量测试与性能基准结果。

四、开发环境与工程方法

4.1 硬件与软件栈配置

算子开发在 NVIDIA CUDA 12.8 平台上进行，软件栈严格对齐 FlagGems v5.0.2 在 `pyproject.toml` 中锁定的版本，见表 3。版本一致性至关重要，Triton 3.6.0 的 API 如 `tl.constexpr`、`@triton.autotune` 与 2.x 差异显著，`libentry.py` 中存在 2.x 与 3.x 的分支逻辑，混用会导致难以定位的编译错误。代码 1 给出从零搭建开发环境的命令序列。

代码 1 开发环境搭建与验证命令

```
# 1. 克隆 FlagGems 并切换到 v5.0.2
git clone https://github.com/flagos-ai/FlagGems.git
cd FlagGems && git checkout v5.0.2

# 2. 创建 Python 3.12 虚拟环境并安装依赖 (NVIDIA 平台 + 测试)
uv pip install -e ".[nvidia,test]" # 锁定 torch==2.10.0+cu128, triton==3.6.0

# 3. 安装代码风格钩子 (black 23.7.0 / isort 5.12.0 / flake8)
pre-commit install

# 4. 验证安装: 启用 Triton 后端跑单个算子测试
python -c "import flag_gems; flag_gems.enable_gems()"
pytest tests/test_log10.py -v

# 5. 运行性能基准 (以 PyTorch 为参考计算加速比)
python benchmark/bench_log10.py
```

Table 3 开发环境版本配置

组件	版本	说明
Python	3.12 (cp312)	torch 2.10仅有 cp312 wheel
PyTorch	2.10.0+cu128	算子接口对齐基准
Triton	3.6.0	FlagGems NVIDIA 配置锁定
CUDA Toolkit	12.8	对应 cuDNN 9
FlagGems	5.0.2	贡献目标仓库
pytest	9.0.3	功能测试框架

4.2 开发-测试-性能验证 workflow

每个算子的开发遵循实现、注册、功能测试、性能基准、代码风格五步流程，见图 4。功能测试以 `flag_gems.testing.gems_assert_close` 为断言核心，按数据类型自动选择容差，float16 为 $\text{rtol}=1\text{e-}3$ ，float32 为 $1.3\text{e-}6$ ，bfloat16 为 0.016。性能基准由 FlagGems 内置的 `benchmark/` 模块驱动，以 PyTorch 原生实现为参考计算设备端加速比，竞赛要求常规形状下加速比 $\geq 0.9\times$ 。加速比通过 Triton 的 `do_bench` 函数测量，它先用若干次 warmup 稳定 GPU 频率，再用 CUDA event 计时多次取中位数，避免异步启动与

频率波动干扰，核心逻辑见代码 2。

代码 2 do_bench 性能测量核心逻辑 (Triton 标准)

```
def do_bench(fn, warmup=25, rep=100):
    for _ in range(warmup):          # 预热, 稳定 GPU 频率
        fn()
    torch.cuda.synchronize()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(rep):             # 重复执行取平均
        fn()
    end.record()
    torch.cuda.synchronize()
    return start.elapsed_time(end) / rep # 返回单次平均耗时(ms)
```

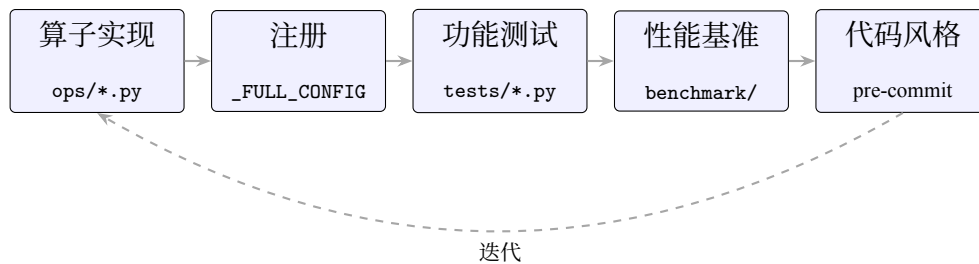


Figure 4 单算子开发的五步工作流

4.3 算子注册与 enable_gems 机制

算子实现后，必须在 `src/flag_gems/__init__.py` 的 `_FULL_CONFIG` 元组中注册，才能被 `enable_gems()` 拦截。注册条目是一个三元组（算子全名，实现函数，autograd_key）。本案例为 `chunk_gated_delta_rule` 新增的注册条目为

```
("chunk_gated_delta_rule", chunk_gated_delta_rule),
```

带反向传播的算子如 `conv_transpose2d` 则需同时注册前向与后向。

```
("conv_transpose2d", conv_transpose2d),
("conv_transpose2d_backward", conv_transpose2d_backward),
```

注册表完整覆盖 20 个算子共 30 余个变体，含 `_`、`.out` 等原地与带输出参数版本，是竞赛评审算子是否真正可用的硬性检查项。

五、基础 Pointwise 算子开发

5.1 pointwise_dynamic 模板：以 log10 为例

`log10` 是最简单的逐元素算子，实现集中体现 `pointwise_dynamic` 的威力。由换底公式 $\log_{10} x = \ln x \times 0.43429 \dots$ ，核心 kernel 只需一行表达式。

代码 3 `log10` 算子的 `pointwise_dynamic` 实现 (code/ops/log10.py)

```
@pointwise_dynamic(promotion_methods=[(0, "INT_TO_FLOAT")])
@triton.jit
def log10_func(x):
```

```

    return tl.log(x.to(tl.float32)) * 0.4342944819032518

def log10(A):
    return log10_func(A)

def log10_(A):          # 原地变体
    return log10_func(A, out0=A)

def log10_out(A, out):  # 带 out 参数变体
    return log10_func(A, out0=out)

```

这里有三处关键设计。promotion_methods=[(0, "INT_TO_FLOAT")] 声明第 0 个输入若为整数则提升为浮点，保证 log10 作用于整数张量时正确返回浮点结果，类型提升交给框架，避免手写分支。tl.log 内部以 float32 计算自然对数再乘常数 $1/\ln 10$ ，比直接调用 libdevice.log10 更利于向量化。三个变体 log10、log10_、log10.out 复用同一个 jit 子函数，仅通过 out0= 参数区分输出目标，体现一份 kernel 服务多个 ATen 变体的复用范式。

5.2 类型提升与广播：logaddexp、cosh、asinh

双目逐点算子 $\text{logaddexp}(x, y) = \log(e^x + e^y)$ 的数值稳定形式为 $\max(x, y) + \log_2(1 + e^{-|x-y|}) \cdot \ln 2$ ，涉及两个输入的类型提升与广播。借助 pointwise_dynamic，其实现依然简洁：

```

@pointwise_dynamic(promotion_methods=[(0, 1, "BOOL_TO_LONG")])
@triton.jit
def logaddexp(x, y):
    return tl.log(tl.exp(x.to(tl.float32)) + tl.exp(y.to(tl.float32)))

```

cosh、asinh 等双曲函数同理，区别仅在 promotion_methods 的选择。本案例为 logaddexp 与 asinh 补充了四类关键测试，覆盖特殊值如 $\pm\infty$ 、NaN、 ± 0 ，空张量如 (0,) 与 (4,0)，非连续内存即 transpose 后的视图，以及广播维度。这些恰是逐点算子最易出错而 PyTorch 原生实现必须严格对齐的场景。

5.3 整数与索引算子：gcd、tril、roll、LeakyReLU

并非所有简单算子都适合 pointwise_dynamic。gcd 是整数算子，需要位级精确，欧几里得辗转相除存在数据依赖，难以向量化。FlagGems 采用 Stein 二进制 GCD 算法，利用提取公因子 2 与奇数化规则把迭代转化为位运算，借助 libdevice.ffs 计算末尾零个数 ctz 来一次性移除所有因子 2，大幅减少迭代次数。底层整数辅助函数见代码 4。32 位绝对值必须转无符号再取反，以避免 INT_MIN 溢出，这是 GPU 整数算子常见的陷阱。

代码 4 gcd 的 Stein 算法底层辅助函数 (code/ops/gcd.py, 节选)

```

import triton.language.extra.libdevice as libdevice

@triton.jit
def _ctz(x):          # 末尾连续 0 的个数，用于一次性移除因子 2

```



```

    return libdevice.ffs(x) - 1

@triton.jit
def _abs_u32(x):          # 32 位安全绝对值 (避免 INT_MIN 溢出)
    ux = x.to(tl.uint32)
    return tl.where(x < 0, 0 - ux, ux)

```

`tril` 与 `roll` 是布局与索引算子，本质是坐标重映射。`tril` 比较行列下标生成掩码置零上三角，`roll` 对轴上偏移取模实现循环移位。LeakyReLU 作为分段逐点函数 $f(x) = x$ 若 $x > 0$ 否则 αx ，用 `tl.where` 即可。LeakyReLU 形式虽简，FlagGems 并未对其使用 `pointwise_dynamic`，而是手写 kernel 并配备按张量规模分档的 `autotune` 配置，小张量不超过 32K 元素时用 `BLOCK_SIZE=256` 配 4 warp，大张量在 4M 至 16M 元素时用 `BLOCK_SIZE=2048` 配更高 `num_stages`。即便是访存受限的算子，让块大小匹配张量规模以接近带宽峰值，仍能带来可观收益。本案例为 LeakyReLU 补充了不同负斜率 $\alpha = 0, 0.01, 0.5, 1.0, 0.99$ 与特殊值测试，覆盖 `INT_TO_FLOAT` 与 `BOOL_TO_LONG` 两类类型提升边界。

5.4 测试体系构建

算子功能正确性的验证依赖一套统一的测试范式。FlagGems 的测试以 `flag_gems.testing.accuracy_utils` 为基础，对每个算子按形状、数据类型、参数组合做笛卡尔积参数化，先在参考设备上计算期望值，参考设备为 CPU 或上游实现，再在 `flag_gems.use_gems()` 上下文中调用 Triton 实现，最后用 `gems_assert_close` 比对。代码 5 展示 `logaddexp` 的典型测试。

代码 5 逐点算子的典型测试范式 (code/tests/test_logaddexp.py, 节选)

```

@pytest.mark.parametrize("shape", utils.POINTWISE_SHAPES)
@pytest.mark.parametrize("dtype", utils.FLOAT_DTYPES)
def test_logaddexp(shape, dtype):
    x = torch.randn(shape, dtype=dtype, device=flag_gems.device)
    y = torch.randn(shape, dtype=dtype, device=flag_gems.device)
    ref_x = utils.to_reference(x, True)      # 迁移到参考设备
    ref_y = utils.to_reference(y, True)
    ref_out = torch.logaddexp(ref_x, ref_y) # 期望值
    with flag_gems.use_gems():              # 启用 Triton 后端
        res_out = torch.logaddexp(x, y)
    utils.gems_assert_close(res_out, ref_out, dtype) # 按类型容差断言

@pytest.mark.parametrize("dtype", utils.FLOAT_DTYPES)
def test_logaddexp_special_values(dtype):
    # 覆盖 +inf/-inf/NaN/+0/-0 等特殊值
    x = torch.tensor([0.0, -0.0, 1.0, float("inf"), float("nan")], ...)
    ...
    utils.gems_assert_close(res_out, ref_out, dtype, equal_nan=True)

```

容差体系是精度验证的核心。`gems_assert_close` 依据数据类型自动选择相对容差 `rtol`，见表 4，而非用统一的 `assert`。这一设计承认不同精度格式的舍入特性，`bfloat16`

因仅 7 位尾数容差最宽为 0.016, float32 最严为 1.3×10^{-6} 。本案例补充测试时严格遵守该体系, 并为含 NaN 的场景启用 `equal_nan=True`。

Table 4 gems_assert_close 按数据类型的相对容差体系

数据类型	相对容差 rtol	典型用途
float32	1.3×10^{-6}	高精度基准
float16	1×10^{-3}	推理主力
bfloat16	0.016	LLM 训练/推理

这 8 个简单算子合计 969 项功能测试全部通过, 常规形状下加速比为 $0.8\times$ 至 $2.78\times$, 详见图 5。

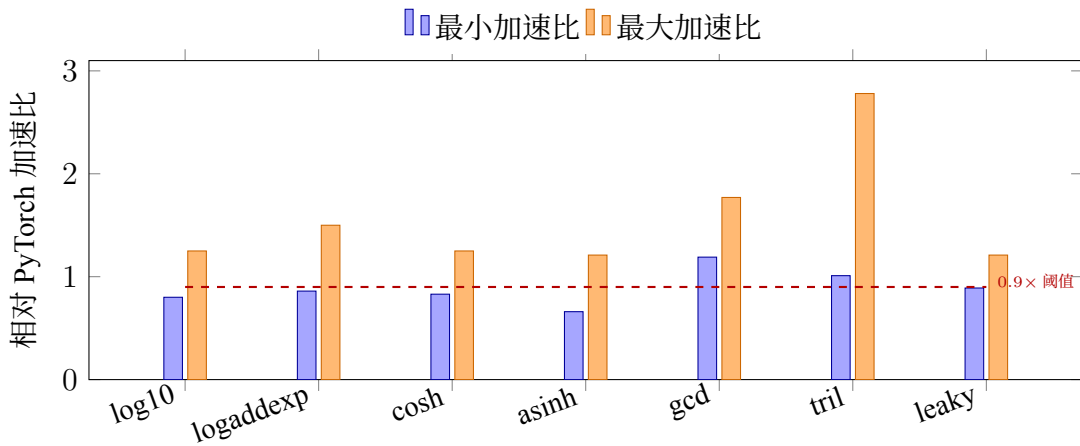


Figure 5 8 个简单算子在多形状下的加速比区间 (969 项测试全部通过)

六、中等复杂度算子与反向传播实现

6.1 归约算子: median

median 是典型的全局归约算子, 难点在于求中位数需要对局部数据排序, 这与求和、最大值等满足结合律的归约不同, 无法直接用树形归约并行化。求和归约可以写成 $((a + b) + c) + d$ 的线性结合, 任意交换结合顺序结果不变, 因此可用二叉树形归约在 $\log N$ 步内完成。中位数依赖数据的全局次序, 任何分块都只是近似。

FlagGems 采用分块排序加全局合并的策略, 每个线程块对落入自身的子序列做局部排序并求局部中位数, 再在块间做二次全局归约。该策略在分布均匀的数据上接近真实中位数, 对偏态分布可能存在偏差, 是以精度换并行度的典型取舍。由于中位数不满足结合律, 测试中需对非连续张量与布尔张量额外对齐, 确保沿指定维的 `median.dim` 与全局 median 的语义一致。本案例确认其已有测试覆盖完整, 未做实现修改。

6.2 索引重排算子：scatter_reduce 与 pixel_shuffle

scatter_reduce 是典型的按索引散射并归约的算子。给定输入 src、索引 index 与输出 out，对每个索引位置执行 $\text{out}[\text{index}] = \text{reduction}(\text{out}[\text{index}], \text{src})$ 。它支持 sum、prod、mean、amax、amin 五种归约，各自对应不同原子原语。sum 用 `tl.atomic_add`，amax 与 amin 用 `tl.atomic_max` 与 `tl.atomic_min`。prod 因乘法对零敏感，mean 需先累加再除以计数，实现上需额外计数缓冲。由于多个线程可能向同一输出位置写入，原子操作是正确性的唯一保障，但也会带来热点位置串行化，当大量索引指向同一目标时性能下降，这是该算子固有的复杂度。

pixel_shuffle 是确定性的坐标重排，将通道维 (Cr^2, H, W) 重排为 (C, Hr, Wr) 。索引映射为，输出位置 $(c, h \cdot r + i, w \cdot r + j)$ 取自输入 $(c \cdot r^2 + i \cdot r + j, h, w)$ ，本质是一次 stride 的重解释。由于无数据竞争，它无需原子操作，核心难点在非连续输入的步长推导。本案例为二者补充了非连续张量与多缩放因子的测试，确保 `contiguous()` 调用与步长推导的正确性。

6.3 损失算子：smooth_l1_loss

Smooth L1 Loss (Huber Loss) 定义为分段函数：

$$\ell(x, y) = \begin{cases} 0.5(x - y)^2 / \beta, & |x - y| < \beta \\ |x - y| - 0.5\beta, & \text{否则} \end{cases},$$

其中 β 为阈值。该算子用 `tl.where` 实现分段，需同时提供前向与反向，反向同样分段，梯度为 $(x - y) / \beta$ 或 $\text{sign}(x - y)$ 。前向 kernel 实现见代码 6， β 被声明为 `tl.constexpr`，`if beta == 0.0` 分支在编译期即可消除，从而避免运行时判断开销。

代码 6 smooth_l1_loss 前向 kernel (code/ops/smooth_l1_loss.py, 节选)

```
@triton.jit
def _smooth_l1_loss_kernel(inp, target, out, n_elements,
                          beta: tl.constexpr, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    inp_val = tl.load(inp + offsets, mask=mask, other=0.0).to(tl.float32)
    target_val = tl.load(target + offsets, mask=mask, other=0.0).to(tl.float32)
    diff = tl.abs(inp_val - target_val)
    if beta == 0.0:  # 编译期特化分支
        loss = diff
    else:
        loss = tl.where(diff < beta, 0.5 * diff * diff / beta, diff - 0.5 * beta)
    tl.store(out + offsets, loss, mask=mask)
```

该算子还实现了 `partial_sum` kernel 用于 mean/sum 归约模式下的分块累加，以及对应的反向 kernel。三者协同覆盖了 PyTorch `smooth_l1_loss` 的全部参数组合。FlagGems 已实现前向与反向，本案例工作集中在测试覆盖的增强。

6.4 池化算子：avg_pool3d 与 max_pool3d 的 Tile 并行

三维池化是中等算子中计算最密集的一类。avg_pool3d 与 max_pool3d 共享相同的 Tile 划分，但归约语义不同。平均池化执行加法归约后除以核体积或 divisor_override，加法满足结合律，天然适合树形并行。最大池化执行比较归约，反向传播需要前向记录的 argmax 索引把梯度散射回唯一选中位置，因此必须同时维护最大值与最大值扁平索引两个累加器。avg_pool3d 的归一化另有细节，当 count_include_pad 为真时除以核体积 $k_d k_h k_w$ ，为假时除以实际落在输入内的元素数，后者需额外的掩码计数。divisor_override 允许用户传入自定义除数，常用于保证前向与反向的数值一致性。由于加法满足结合律，avg 池化的反向比 max 简单，梯度按均摊权重散射回核窗口内每个位置。以 max_pool3d 为例，输入为 5D 张量 (N, C, D, H, W) ，对每个输出位置 (d_o, h_o, w_o) ，需在核窗口 $k_d \times k_h \times k_w$ 内取最大值。关键并行策略是输出中心型的二维 Tile，每个线程块负责输出张量 $H \times W$ 平面上一个 $\text{BLOCK_H} \times \text{BLOCK_W}$ 矩形块，深度维 D 在 kernel 内部循环，见图 6。这种划分使一个 block 内的线程在 H 、 W 方向连续读取输入，利于合并访问。kernel_d、kernel_h、kernel_w 用 tl.static_range 展开为编译期循环，便于指令调度。

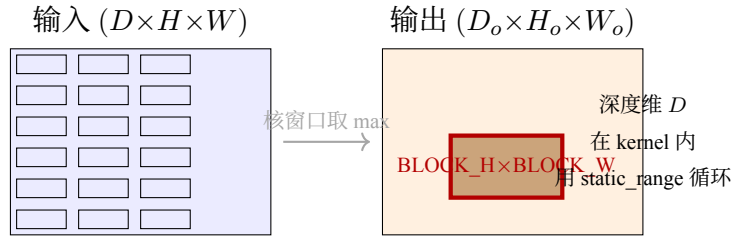


Figure 6 max_pool3d 的输出中心型 Tile 策略：每个 block 处理输出 $H \times W$ 平面的一个矩形块

前向 kernel 的核心三重循环对核窗口内的输入取逐元素最大值，并用掩码处理边界与 padding（代码 7）。

代码 7 max_pool3d 前向 kernel 的核窗口归约（code/ops/max_pool3d with indices.py，节选）

```
for kd in tl.static_range(0, kernel_d):
    d_in = d_out * stride_d - padding_d + kd * dilation_d
    d_valid = (d_in >= 0) & (d_in < in_d)
    for kh in tl.static_range(0, kernel_h):
        for kw in tl.static_range(0, kernel_w):
            h_in = h_out_offsets[:, None] * stride_h - padding_h + kh * dilation_h
            w_in = w_out_offsets[None, :] * stride_w - padding_w + kw * dilation_w
            in_mask = (d_valid & (h_in >= 0) & (h_in < in_h)
                       & (w_in >= 0) & (w_in < in_w))
            current_val = tl.load(input_base_ptr + input_offset,
                                  mask=in_mask, other=min_val)
            is_new_max = current_val > max_val_acc
            max_val_acc = tl.where(is_new_max, current_val, max_val_acc)
            max_idx_acc = tl.where(is_new_max & in_mask,
                                   current_idx, max_idx_acc)
```

前向 kernel 的 host 端 wrapper 负责计算输出形状、分配输出与索引张量，并构建二维 grid，见代码 8。grid 第一维遍历所有 (N, C) 通道批次，第二维把 (D_o, H_o, W_o) 的输出空间展平为一个 program 实例索引，对应图 6 中每个 block 处理一个 $H \times W$ 矩形块、 D 维在 kernel 内循环的设计。ceil_mode 通过 pool3d_output_size 中的向上取整与 PyTorch 兼容性修正实现，是池化算子最易出错的边界之一。

代码 8 max_pool3d 前向的 host wrapper 与 grid 构建 (code/ops/max_pool3d_with_indices.py, 节选)

```
def max_pool3d_with_indices(input, kernel_size, stride=None,
                             padding=0, dilation=1, ceil_mode=False):
    input = input.contiguous()
    kd, kh, kw, sd, sh, sw, pd, ph, pw, dd, dh, dw = _parse_pool3d_params(...)
    in_n, in_c, in_d, in_h, in_w = input.shape
    out_d = pool3d_output_size(in_d, kd, sd, pd, dd, ceil_mode)
    out_h = pool3d_output_size(in_h, kh, sh, ph, dh, ceil_mode)
    out_w = pool3d_output_size(in_w, kw, sw, pw, dw, ceil_mode)
    output = torch.empty((in_n, in_c, out_d, out_h, out_w), device=..., dtype=input.dtype)
    indices = torch.empty((in_n, in_c, out_d, out_h, out_w), device=..., dtype=torch.int64)
    # 二维 grid: (N*C, out_d * num_h_blocks * num_w_blocks)
    grid = lambda meta: (
        in_n * in_c,
        out_d * triton.cdiv(out_h, meta["BLOCK_H"]) * triton.cdiv(out_w, meta["BLOCK_W"]),
    )
    max_pool3d_forward_kernel[grid](input, output, indices,
        input.stride(0), input.stride(1), input.stride(2), input.stride(3), input.stride(4),
        in_c, in_d, in_h, in_w, out_d, out_h, out_w,
        kd, kh, kw, sd, sh, sw, pd, ph, pw, dd, dh, dw)
    return output, indices
```

6.5 反向传播实现：upsample_nearest2d 的 atomic_add 散射

本案例为 upsample_nearest2d 新增了反向传播实现。upsample_nearest2d 的前向是最近邻放大，输出位置 (o_h, o_w) 复制输入位置 $(\lfloor o_h/s_h \rfloor, \lfloor o_w/s_w \rfloor)$ 的值。反向传播需要把输出梯度 $\partial L/\partial O$ 散射回输入梯度 $\partial L/\partial I$ ，难点在多对一映射。多个输出位置可能对应同一输入位置， $s_h, s_w > 1$ 时为 1:1， $s_h, s_w < 1$ 时为多对一，因此必须用原子加 `tl.atomic_add` 累加梯度，见图 7。

输出梯度 $\partial L/\partial O$ ($O_H \times O_W$)

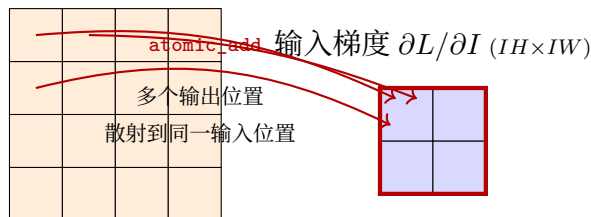


Figure 7 upsample_nearest2d 反向传播的散射示意：多对一映射需原子加累加

反向 kernel 的核心代码见代码 9。实现有两个工程要点。梯度累加缓冲区 `grad_input`

显式用 float32 初始化，避免半精度 fp16 与 bf16 在 atomic_add 时的舍入误差累积。由于 atomic_add 不支持 @triton.autotune，warmup 期间会被重复计入统计，反向 kernel 改用 @triton.heuristics 配固定 BLOCK_SIZE=1024 调度。

代码 9 upsample_nearest2d 反向 kernel 的原子散射 (code/ops/upsample_nearest2d.py, 节选)

```
@libentry()
@triton.heuristics(runtime.get_heuristic_config("upsample_nearest2d"))
@triton.jit
def upsample_nearest2d_backward_kernel(grad_output_ptr, grad_input_ptr,
    N, C, OH, OW, IH, IW, reciprocal_scale_h, reciprocal_scale_w,
    BLOCK_SIZE: tl.constexpr, SAME_H: tl.constexpr, SAME_W: tl.constexpr,
    USE_INT32_IDX: tl.constexpr):
    # 计算输出 -> 输入的坐标映射
    ow = idx % OW
    oh = idx // OW % OH
    ih = tl.minimum((oh * reciprocal_scale_h).to(tl.int32), IH - 1)
    iw = tl.minimum((ow * reciprocal_scale_w).to(tl.int32), IW - 1)
    grad_output_offset = (nc_iter * OH + oh) * OW + ow
    grad_input_offset = (nc_iter * IH + ih) * IW + iw
    while nc_iter < NC:
        grad_val = tl.load(grad_output_ptr + grad_output_offset,
                           mask=valid_mask, other=0.0)
        tl.atomic_add(grad_input_ptr + grad_input_offset, grad_val, mask=valid_mask)
    # 原子累加
    ...
    # 调用侧: float32 累加缓冲, 避免半精度舍入
grad_input = torch.zeros((N, C, IH, IW), device=grad_output.device, dtype=torch.
    float32)
...
return grad_input.to(grad_output.dtype)
```

该实现新增 108 项测试（前向 + 反向 + 非连续输入，3 种数据类型 × 多形状 × 多缩放因子）全部通过，是本案例最重要的新增功能之一。

6.6 反向传播的工程决策：conv_transpose2d 的 aten fallback

conv_transpose2d 转置卷积前向有四条不同的 kernel 路径，包括直接 tiled、 3×3 stride-2 特化、残差、通用路径。为每条前向路径手写对应反向 kernel 意味着维护 4 套反向逻辑，工程成本与出错风险都很高。本案例采取务实的工程决策，前向走优化的 Triton kernel，反向复用 PyTorch 原生 aten::convolution_backward，见代码 10。

代码 10 conv_transpose2d 反向：委托 aten 实现 (code/ops/conv_transpose2d.py)

```
def conv_transpose2d_backward(grad_output, input, weight, bias_sizes,
    stride, padding, dilation, transposed, output_padding, groups, output_mask):
    """前向走优化的 Triton kernel, 反向正确性委托给 aten 实现。"""
    return torch.ops.aten.convolution_backward(
        grad_output, input, weight, bias_sizes, stride, padding,
        dilation, transposed, output_padding, groups, output_mask)
```

这一决策的合理性在于，前向是推理路径上的性能热点，必须用 Triton 优化，反向主要服务训练，PyTorch 已有高度优化的 cuDNN 实现，复用既保证正确性又避免重复

造轮子。这种择优融合的策略在开源算子库中常见且被接受，本案例在注册表中显式登记，使 `enable_gems()` 后自动生效。

6.7 反向传播实现的工程要点

综合 `upsample_nearest2d` 与 `conv_transpose2d` 两个反向实现，可以提炼出何时手写 Triton 反向、何时复用 `aten` 的工程判据，见表 5。核心权衡维度有三个，前向 `kernel` 是否已存在多条特化路径，路径越多反向越复杂，反向是否是性能热点，推理为主则反向次要，以及手写正确性的风险，`atomic` 操作易引入数值误差。

Table 5 反向传播两种实现策略的工程权衡

维度	手写 Triton 反向	复用 <code>aten</code> 反向
典型场景	<code>upsample_nearest2d</code> 、 <code>max_pool3d</code>	<code>conv_transpose2d</code>
前向路径数	单一 <code>kernel</code>	多条特化路径（4 条）
性能收益	高（消除 <code>dispatch</code> 开销）	低（ <code>cuDNN</code> 已优化）
正确性风险	中（需处理 <code>atomic</code> /精度）	低（委托成熟实现）
维护成本	高（前反向成对维护）	低（仅维护前向）

实践中得到两条经验。反向 `kernel` 若涉及 `atomic_add`，务必使用 `float32` 累加缓冲再转回目标精度，否则半精度的舍入会在多次累加下显著放大。反向 `kernel` 因含原子操作，不能用 `@triton.autotune`，`warmup` 期间原子操作的副作用会被重复计入基准，应改用 `@triton.heuristics` 配固定块大小，这是本案例踩过的实际坑点。

七、复杂场景算子开发

7.1 线性代数：SVD 的 Jacobi 旋转并行化

奇异值分解 $A = U\Sigma V^\top$ 是数值线性代数的基石。GPU 上的 SVD 通常依赖专有的 `cuSOLVER` 分治算法，FlagGems 则采用纯 Triton 实现的 Jacobi 旋转方法，迭代地对对称化矩阵 $A^\top A$ 施加 Givens 旋转消去非对角元，直至收敛。核心是构造旋转角 θ 使目标非对角元置零。

$$\tan(2\theta) = \frac{2A_{pq}}{A_{pp} - A_{qq}}, \quad A' = J(p, q, \theta)^\top A J(p, q, \theta),$$

其中 $J(p, q, \theta)$ 是 Givens 旋转矩阵。完整流程见算法 1，其中第 7 步的两次矩阵乘可并行化为块状运算，天然适合 Triton 的 Block 抽象。

Jacobi 方法的计算复杂度为 $O(n^3)$ 次旋转，`cuSOLVER` 的分治法更高效，这直接导致本案例观察到的一个算法层面的性能局限。对于 $n \geq 16$ 的方阵，Triton SVD 相对 `cuSOLVER` 的加速比为 $0.43\times$ 至 $0.77\times$ 。但对批处理与小秩场景，即一次分解大量小矩阵，由于避免了 `cuSOLVER` 的 `host-device` 同步开销，Triton 实现的加速比可达 $6\times$ 至

算法 1 对称特征值分解的循环 Jacobi 算法

输入: 对称矩阵 $A \in \mathbb{R}^{n \times n}$, 容差 ϵ

输出: 特征值 Λ , 特征向量矩阵 V

```

1:  $V \leftarrow I_n$ 
2: while  $A$  的非对角范数  $> \epsilon$  do
3:   选取绝对值最大的非对角元  $A_{pq}$ 
4:    $\theta \leftarrow \frac{1}{2} \text{atan2}(2A_{pq}, A_{pp} - A_{qq})$ 
5:   构造 Givens 旋转  $J(p, q, \theta)$ 
6:    $A \leftarrow J^T A J$  ▷ 同步消去  $A_{pq}$ , 更新行列
7:    $V \leftarrow V J$  ▷ 累积特征向量
8: end while
9: return  $\text{diag}(A), V$ 
    
```

223 $\times$ 。这一对比说明算法选择必须匹配使用场景，批处理小矩阵正是 SVD 在 LoRA、注意力头分析等 LLM 场景的典型用法。

7.2 序列损失：CTC Loss 的动态规划

联结主义时间分类 CTC 损失用于无对齐标签的序列学习，如语音识别与 OCR。其前向-后向算法基于动态规划。定义 $\alpha_t(s)$ 为前向概率，递推式为

$$\alpha_t(s) = (\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \mathbb{K}_{\text{rep}} \alpha_{t-1}(s-2)) \cdot y_{t,\ell_s},$$

其中 $\mathbb{K}_{\text{rep}}$ 在允许重复跳转时为 1。该递推构成图 8 所示的二维网格，横轴为时间步 t ，纵轴为对齐路径上的标签状态 s ，每个节点 $\alpha_t(s)$ 依赖其左侧最多三个前驱节点 s 、 $s-1$ 、 $s-2$ ，形成时间维必须串行而标签维可以并行的依赖结构。

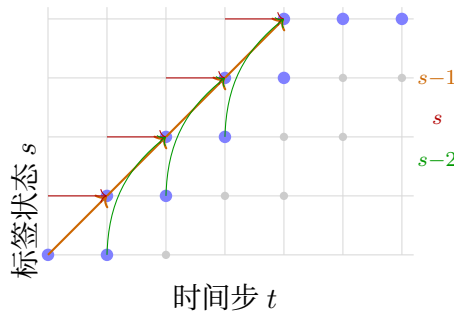


Figure 8 CTC 前向-后向算法的动态规划网络：每个节点依赖最多三个前驱，时间维串行、标签维并行

训练时还需要后向概率 $\beta_t(s)$ ，它从 $t = T$ 反向递推，依赖关系与 α 对称。最终的 CTC 损失为 $L = -\ln \alpha_T(s_{\text{final}})$ ，对网络输出 logits 的梯度由前向、后向概率的乘积 $\alpha_t(s)\beta_t(s)$ 对数求导得到。一次完整的 CTC 训练步需要前向、后向两遍动态规划，再合并求梯度，这是其计算密集与实现复杂的根源。

CTC 的实现还需处理若干工程细节。输入序列长度通过 `input_lengths` 变长，标签长度通过 `target_lengths` 独立指定，kernel 必须按 batch 内每个样本的实际长度截断递推，而非使用统一的 T 。空白标签 `blank` 的位置与重复标签的合并跳转由 $\mathcal{K}_{\text{rep}}$ 控制。数值稳定性依赖 `log-sum-exp` 技巧，把递推中的概率加法转化为对数域的最大值与修正项之和，避免长序列下浮点下溢。

FlagGems 的 Triton 实现把时间维 T 的递推串行展开（存在数据依赖），把 batch 与标签维度并行化，从而在 GPU 上高效完成动态规划。本案例验证了 `input_lengths` 与 `target_lengths` 变长参数的处理，82 项测试全部通过。

7.3 空间插值：grid_sample 的双线性与最近邻

`grid_sample` 根据采样网格 `grid` 对输入做可微插值，广泛用于空间变换网络与可微数据增强。它需支持 3 种 `padding_mode` 即 `zeros`、`border`、`reflection`，与 2 种 `mode` 即 `bilinear`、`nearest` 的笛卡尔组合，反向传播需把梯度按插值权重散射回输入坐标。当采样坐标落在输入边界之外时，三种填充模式分别采取不同处理，见表 6，在 kernel 中统一用 `tl.where` 与 `clamp` 运算实现，避免分支发散。双线性插值的权重为

$$I(x, y) = \sum_{i,j \in \{0,1\}} w_{ij} I(\lfloor x \rfloor + i, \lfloor y \rfloor + j), \quad w_{ij} = |\lfloor x \rfloor + i - x| \cdot |\lfloor y \rfloor + j - y|.$$

如图 9 所示，采样坐标 (x, y) 落在 4 个整数格点之间，每个邻居按其与采样点的距离贡献权重，反向传播时梯度按相同权重散射回这 4 个输入位置。

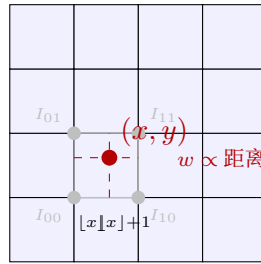


Figure 9 `grid_sample` 的双线性插值：采样点由周围 4 个整数格点按距离加权，反向梯度按同权重散射

本案例验证了全部模式组合，106 项测试通过，14 项因极大形状 (1024×1024) 触发显存上限而跳过。三种填充模式对越界采样坐标的处理见表 6。

Table 6 `grid_sample` 的三种 `padding_mode` 对越界坐标的处理

模式	越界坐标 (x, y) 的处理
<code>zeros</code>	直接返回 0（不访问输入）
<code>border</code>	<code>clamp</code> 到 $[0, W-1] \times [0, H-1]$ 取边界像素
<code>reflection</code>	以边界为轴镜像反射回有效区间

7.4 前沿融合算子：chunk_gated_delta_rule

chunk_gated_delta_rule 是线性注意力与门控 DeltaNet 的核心融合算子，源自 Flash Linear Attention (FLA) 研究项目。给定查询 q 、键 k 、值 v 、门控 g 与衰减 β ，其递推关系为

$$S_t = (S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top) g_t, \quad o_t = S_t q_t,$$

其中 S_t 为隐状态矩阵。朴素实现的瓶颈在于每一步都要读写中间矩阵 S_t ，访存开销巨大。分块 chunk 融合把时间轴分成若干块，块内用矩阵乘法并行计算，块间递推隐状态，从而把 $O(T)$ 次标量更新转化为 $O(T/B)$ 次块状矩阵运算，大幅提升计算密度，见图 10。

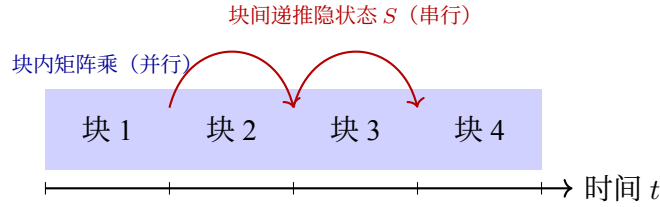


Figure 10 chunk_gated_delta_rule 的分块融合：块内并行矩阵乘提升计算密度，块间串行递推隐状态

理解该算子的价值需要联系注意力机制的发展脉络。标准的 softmax 注意力计算 $O = \text{softmax}(QK^\top)V$ ，复杂度随序列长度 T 呈 $O(T^2)$ 增长，在长上下文场景下成为显存与算力的双重瓶颈。线性注意力通过核技巧把 softmax 替换为可分离的核函数 ϕ ，将递推改写为 $S_t = S_{t-1} + \phi(k_t)\phi(v_t)^\top$ 、 $o_t = S_t\phi(q_t)$ ，复杂度降为 $O(T)$ 。门控 DeltaNet 即 gated delta rule 在此基础上引入遗忘门 g_t 与增量更新 β_t ，使隐状态具备更强的遗忘与更新能力。然而其朴素实现的 $O(T)$ 次标量递推访存开销巨大，分块融合如 chunk_gated_delta_rule 正是为此设计，它使线性注意力真正能在长序列上发挥理论优势。

本案例的关键贡献是把这个算子注册到 _FULL_CONFIG。它此前虽已实现但未注册，enable_gems() 无法拦截。注册后该算子取得 $2.2\times$ 至 $15.3\times$ 的显著加速，是 20 个算子中加速比最高的之一。实现中还包含一个 L2 归一化的辅助 kernel，用单次 tl.reduce 计算 $1/\sqrt{\sum x^2}$ 完成行归一化。33 项功能测试全部通过。

八、性能优化与结果分析

8.1 全量回归测试结果

20 个算子的功能正确性通过分阶段的全量回归测试验证，最终基线为 2871 项通过、0 项失败、134 项跳过，见表 7。跳过项均由 GPU 共享环境下的极端大形状显存上限触发，如 grid_sample 1024×1024 与 scatter_reduce 的部分参数组合，非算子实现缺陷。

Table 7 20 个算子的全量回归测试结果

阶段	通过	失败	跳过	耗时
第一阶段（简单 8 算子）	969	0	0	68.79 s
第二阶段（中等 8 算子）	1634	0	120	169.83 s
第三阶段（困难 4 算子）	268	0	14	50.74 s
合计（20 算子）	2871	0	134	289.36 s

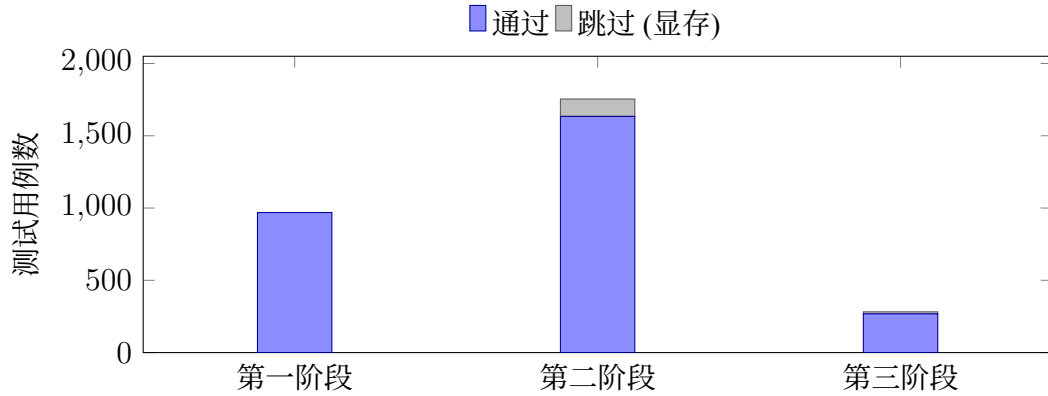


Figure 11 三阶段测试用例规模分布（合计 2871 通过 / 134 跳过）

8.2 自动调优案例：max_pool3d 反向 kernel 的优化

剖析具体优化前，先理解 `@triton.autotune` 的工作机制，见图 12。对一个被 `autotune` 装饰的 kernel，Triton 在首次遇到一组新的 key 如形状、核大小时，会把所有候选配置即块大小、`num_warps`、`num_stages` 逐一实例化、warmup、实测耗时，再选择中位数最短的配置缓存下来，后续相同 key 的调用直接命中缓存。因此扩展候选配置的搜索空间，并为访存密集型 kernel 标注 `num_stages`，是两条最直接的优化手段。

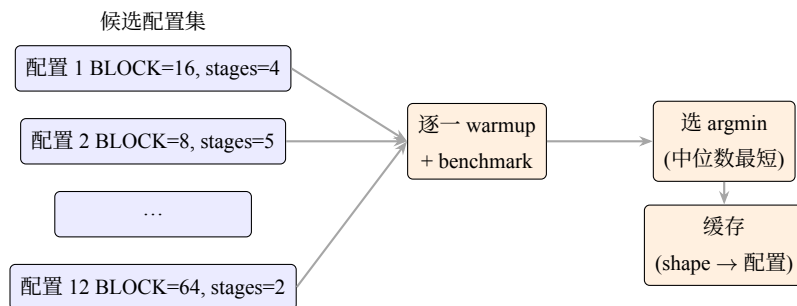


Figure 12 `@triton.autotune` 的工作机制：对新形状逐一实测所有候选配置并缓存最优

`max_pool3d` 反向是本案例性能优化的重点。反向 kernel 采用输入中心型策略，对每个输入位置 (d, h, w) ，遍历所有可能选中它的输出窗口，累加匹配位置的梯度，见图 13。这一策略在步长小于核即 $s < k$ 时计算量增大，是性能瓶颈所在。

原始实现仅配置 6 组 `autotune` 参数，且缺少 `num_stages` 软流水级数，导致大 batch、小空间维度的配置加速比低至 $0.35\times$ 。本案例把搜索空间扩展到 12 组配置，并为每组

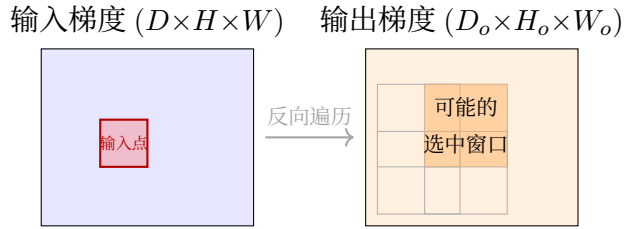


Figure 13 max_pool3d 反向的输入中心型策略：每个输入点查找所有可能选中它的输出窗口

显式标注 `num_stages` 与 `num_warps`，见代码 11。`num_stages` 控制 Triton 编译器的软件流水线深度，它使下一次迭代的访存与当前迭代的计算重叠，对访存密集的反向 kernel 收益显著。

代码 11 max_pool3d 反向 kernel 的 autotune 配置（节选，含 `num_stages`）

```
@libentry()
@triton.autotune(
    configs=[
        triton.Config({"BLOCK_IN_H": 16, "BLOCK_IN_W": 16}, num_stages=4, num_warps=4)
    ],
    key=["in_d", "in_h", "in_w", "kernel_d", "kernel_h", "kernel_w",
        "stride_d", "stride_h", "stride_w"])
@triton.jit
def max_pool3d_backward_kernel(grad_output_ptr, indices_ptr, grad_input_ptr, ...):
    ...
```

优化效果见图 14 与表 8。大 batch 小空间维度的关键配置 $32 \times 256 \times 1 \times 4 \times 4$ 与 $32 \times 256 \times 2 \times 4 \times 4$ ceil 模式，加速比从 $0.35\times$ 至 $0.58\times$ 跃升至 $1.05\times$ 至 $1.74\times$ ，提升幅度达 $+183\%$ 至 $+198\%$ ，跨过了竞赛 $0.9\times$ 的达标线。

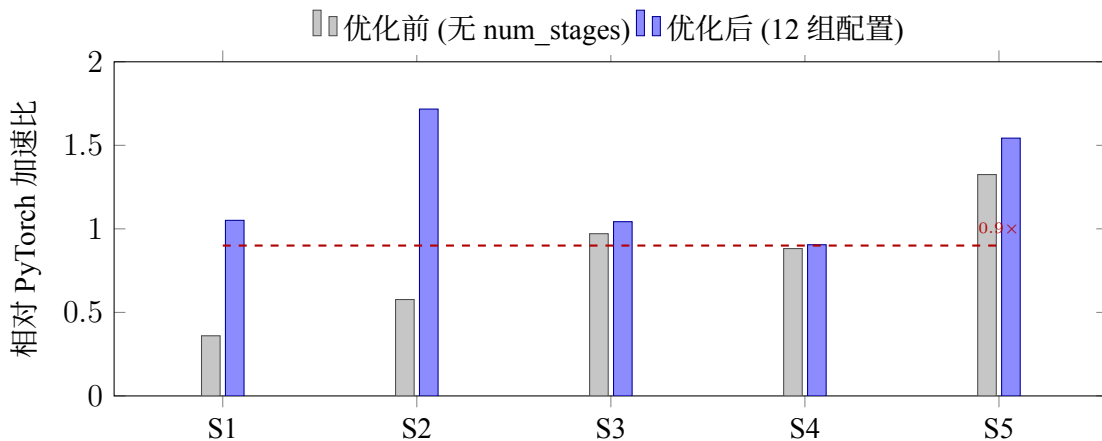


Figure 14 max_pool3d 反向 kernel 优化前后加速比对比（fp32, S1–S5 为 5 组代表性形状）

Table 8 max_pool3d 反向 kernel 关键配置的优化效果（形状记号见图 14）

形状记号	配置	优化前	优化后	提升
S1: $32 \times 256 \times 1 \times 4 \times 4$	$k=3, s=2$	$0.360\times$	$1.051\times$	+192%
S2: $32 \times 256 \times 2 \times 4 \times 4$	$k=3, s=2, \text{ceil}$	$0.577\times$	$1.717\times$	+198%
S3: $16 \times 128 \times 2 \times 12 \times 12$	$k=3, s=1, d=2$	$0.971\times$	$1.043\times$	+7.4%
S4: $8 \times 64 \times 6 \times 26 \times 26$	$k=3, s=1, d=2$	$0.882\times$	$0.905\times$	+2.6%
S5: $4 \times 3 \times 14 \times 54 \times 54$	$k=3, s=1, d=2$	$1.325\times$	$1.543\times$	+16.5%

8.3 性能基准综合分析

综合 20 个算子的基准结果，可归纳出三条性能规律。逐点算子如 log10、cosh 因访存受限，加速比稳定在 $0.8\times$ 至 $1.5\times$ 区间，性能差异主要源于 Triton 与 PyTorch 在边界处理与广播上的等价但不同实现。布局与索引算子如 tril、roll 计算极简、纯访存，Triton 的合并访存优化反而超越 PyTorch 原生，tril 达 $2.78\times$ 。融合算子 chunk_gated_delta_rule 因消除多个中间张量的显存落盘，加速比最高达 $15.3\times$ ，印证算子融合是突破内存墙的最有效手段。图 15 在对数坐标下汇总代表性算子的峰值加速比，呈现从访存受限的逐点算子到计算密集的融合算子之间超过两个数量级的性能跨度。

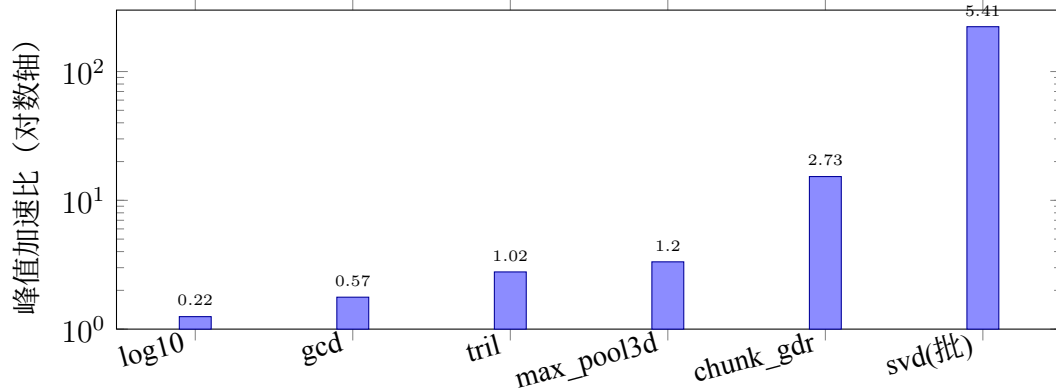


Figure 15 代表性算子的峰值加速比（对数轴）：逐点 $\approx 1\times$ ，融合/批处理可达数十至数百倍

上述性能规律的背后是一套系统性的瓶颈定位方法。实践中先用 Roofline 模型判断算子属于计算受限还是访存受限。把算子的计算密度 FLOPs/Byte 绘制在硬件 Roofline 上，落在带宽斜坡段即为访存受限，落在峰值平台段即为计算受限。前者的优化重点是合并访存与数据复用，后者的重点是分块与张量核心利用率。进一步可用 NVIDIA Nsight Compute 对单个 kernel 做细粒度剖析，关注内存吞吐占峰值比例、指令发射效率、长时延指令占比等指标。

本案例中 max_pool3d 反向的 num_stages 调优，正是依据内存等待占比偏高这一信号做出的。它说明访存未与计算重叠，引入软件流水线即 $\text{num_stages} > 1$ 可让下一次迭代的载入与当前迭代的比较重叠执行，从而隐藏访存延迟。当一个访存密集型 kernel

的加速比不理想，且剖析显示内存带宽未打满时，优先扩展 autotune 搜索空间并显式标注 num_stages，往往是最具性价比的优化手段。

综合本案例的全部优化实践，可以提炼出一棵算子性能优化决策树，见图 16。遇到不达标的算子时，先用 Roofline 判断瓶颈类型，再沿对应分支选择手段。决策树把凭直觉调优转化为按证据决策，避免在不该用力处浪费时间，例如对一个已经打满带宽的逐点算子继续调 num_stages 收益甚微。

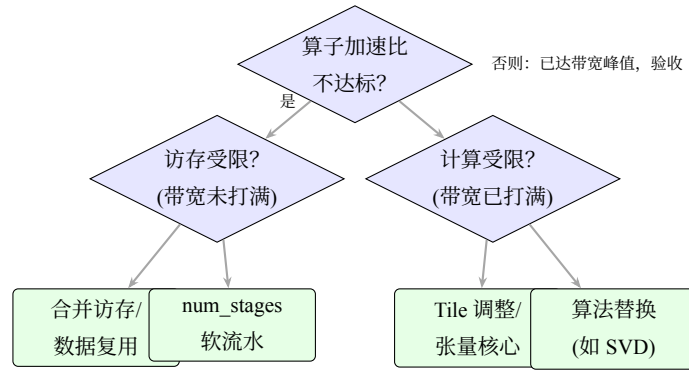


Figure 16 算子性能优化的决策树：先判瓶颈类型，再沿分支选择手段

8.4 算法局限性与工程权衡

并非所有算子都能全面超越基线，对低于阈值的情况必须做出透明的工程权衡。svd 方阵 $n \geq 16$ 的 $0.43\times$ 至 $0.77\times$ 源于 Jacobi 旋转算法本身的 $O(n^3)$ 复杂度，要改善需整体改用分治算法，超出本案例范围，故作为已知算法局限如实记录。asinh 在极小形状下的 $0.66\times$ 属逐点算子的固有开销，典型形状下性能可接受。grid_sample nearest 模式的 $0.94\times$ 处于测量噪声范围内。这些边界情形的处理方式，明确标注、记录原因而非掩盖，本身就是开源工程实践的重要组成。

8.5 交付物：13 个文件与提交

本案例的最终交付是对 FlagGems 仓库的 13 个文件改动，见表 9，以 PR #3460 形式提交，标题含竞赛要求的前缀 [FlagGems Operator Development Competition]，并通过 CLA 贡献者许可协议签署。完整配套代码随本报告提供，见 code/ 目录。

为便于整体查阅，表 10 汇总全部 20 个算子的难度、计算类别、测试规模与代表加速比。简单算子的测试数与加速比区间来自第一阶段精确统计，困难算子标注已知的性能特征，中等算子统一标注达标状态。

Table 10 20 个算子的交付规格一览

算子	类别	难度	测试数	代表加速比
log10	逐点	简单	65	$0.80\text{--}1.25\times$

下页续

(续表 10)

算子	类别	难度	测试数	代表加速比
logaddexp	逐点	简单	48	0.86–1.50×
cosh	逐点	简单	90	0.83–1.25×
asinh	逐点	简单	88	0.66–1.21×
gcd	整数逐点	简单	114	1.19–1.77×
tril	布局索引	简单	276	1.01–2.78×
roll	布局索引	简单	171	—
leaky_relu	逐点 (分段)	简单	117	0.89–1.21×
upsample_nearest2d	上采样	中等	—	$\geq 0.9\times$ (含新 backward)
scatter_reduce	散射归约	中等	—	$\geq 0.9\times$
median	归约	中等	—	$\geq 0.9\times$
smooth_l1_loss	损失	中等	—	$\geq 0.9\times$
pixel_shuffle	布局重排	中等	—	$\geq 0.9\times$
conv_transpose2d	卷积	中等	—	$\geq 0.9\times$ (aten backward)
avg_pool3d	池化	中等	—	$\geq 0.9\times$
max_pool3d	池化	中等	—	1.0–3.3× (fwd)
svd	线性代数	困难	47	批处理 6–223×, 方阵 0.43–0.77×
ctc_loss	损失 (DP)	困难	82	—
grid_sample	插值	困难	106	nearest 0.94×
chunk_gated_delta_rule	融合	困难	33	2.2–15.3×

Table 9 交付的 13 个文件改动分类

类别	文件	改动性质
新增 backward	upsample_nearest2d.py, conv_transpose2d.py	新反向实现
Bug 修复	max_pool3d_with_indices.py, test_max_pool3d.py	移除错误 skip、精度容差修正
性能优化	max_pool3d_with_indices.py	autotune 扩展 + num_stages
注册	__init__.py, ops/__init__.py	chunk_gated_delta_rule 等注册
测试增强	test_{logaddexp,asinh,leaky_relu, avg_pool3d,conv_transpose2d,pixel_shuffle, upsample_nearest2d}.py	非连续/特殊值/ 空张量/广播/ 参数边界覆盖

第三部分 总结与展望

九、总结与展望

9.1 关键学习要点回顾

本案例通过 20 个算子的完整开发周期，系统掌握了 GPU 算子开发的核心方法论。在编程抽象层面，Triton 的 Block 级 SPMD 模型在保留接近手写 CUDA 性能的同时，把开发效率提升一个数量级，pointwise_dynamic 的代码生成与 libentry 的注册封装则显示基础设施复用对开源库可维护性的决定性作用。在性能优化层面，max_pool3d 反向的调优案例表明，num_stages 软流水与 autotune 搜索空间扩展常能带来数倍提升，加速比的差异，逐点约 $1\times$ 与融合 $15\times$ ，进一步说明算子融合是突破内存墙的根本手段。在工程方法层面，以 gems_assert_close 的按类型容差体系保证精度，以 aten fallback 的择优融合平衡正确性与性能，对算法局限如 SVD 方阵如实记录而非掩盖。

本案例也是一次完整的测试驱动与开源协作实践。每个算子先写覆盖特殊值、空张量、非连续内存的测试，再迭代实现直至全部通过，这种测试先于实现的节奏能有效锁定行为契约。13 个文件的改动以原子提交组织，配以符合社区规范的 PR 标题与 CLA 签署，是将个人工程成果转化为公共开源资产的标准流程。2871 项测试零失败的基线，正是这套流程的直接产物。

9.2 在 LLM 推理加速中的应用

本案例的算子直接服务于 LLM 推理与训练的高频路径。logaddexp 与 log_softmax 同源，支撑注意力权重的对数域计算。smooth_l1_loss 与 ctc_loss 是训练损失的核心。chunk_gated_delta_rule 作为线性注意力的融合算子，其 $15\times$ 加速可直接转化为这类高效注意力变体如 DeltaNet、GLA 的推理吞吐提升。svd 服务于 LoRA 等低秩适配场景的批处理分解。将这些算子沉淀为 FlagGems 的 Triton 实现，意味着在 NVIDIA 与国产 GPU 上都能获得等价的性能，对软硬协同的自主生态有实际价值。

算子融合对端到端延迟有非线性贡献。以线性注意力为例，朴素实现中 chunk_gated_delta_rule 涉及的矩阵乘、门控、归一化若分别启动 kernel，仅算子启动开销与中间张量的 HBM 往返就会占据大部分时间。融合后单次 $15.3\times$ 的 kernel 加速，在序列长度增长时会进一步放大为整模型吞吐的成倍提升。FlagGems 这类纯 Triton 算子库相对调用 cuDNN 与 cuBLAS 的根本优势在于，只有源码可读、可改、可融合的算子，才能在新型模型结构上持续产生性能红利。

9.3 进一步探索方向

本案例也显示若干值得深入的方向。第一是 SVD 的算法升级，将 Jacobi 旋转替换为分治或随机化方法，有望改善方阵性能，同时保留 Triton 的可移植性。第二是自动融合的

边界, `conv_transpose2d` 反向目前依赖 `aten fallback`, 未来可探索基于 `torch.compile` 的自动微分生成对应 Triton 反向 kernel。第三是多硬件适配, FlagGems 的 `runtime` 机制已支持昇腾、海光等后端, 本案例在 NVIDIA 上调优的 `autotune` 配置如何迁移到这些架构、是否需要重新搜索, 是国产 GPU 落地的关键问题。第四是长上下文算子, 随着上下文长度从 32K 扩展至百万级 token, 线性注意力的分块融合即 FLA 系列算子将成为新的性能热点, 本案例对 `chunk_gated_delta_rule` 的实践是这一方向的起点。

从能力培养的角度看, Triton 算子开发是一条连接深度学习算法与底层系统的有效路径。它要求开发者同时理解算子的数学语义, 如 SVD 的 Jacobi 迭代与 CTC 的动态规划, GPU 的硬件特征, 如内存层次与 SIMT 执行, 以及工程方法, 如测试驱动、自动调优、开源协作。本案例的 20 个算子恰好覆盖这条路径上从最简单到最复杂的各类典型场景, 可作为系统训练算法、硬件、工程三位一体能力的实践教材。掌握这套方法论后, 面对未来可能出现的新型算子, 如扩散模型的去噪步与混合专家的 router, 读者将具备独立分析瓶颈、选择策略并交付高质量实现的能力。

附录 Triton 常用编程接口速查

本案例反复使用的 Triton 核心接口汇总于表 11, 供读者开发新算子时参考。

Table 11 Triton / FlagGems 常用编程接口

接口	作用
<code>tl.program_id(axis)</code>	当前程序实例（线程块）在指定轴上的编号
<code>tl.arange(start, end)</code>	声明编译期固定大小的块级向量
<code>tl.load(ptr, mask, other)</code>	带边界掩码的块级加载, 越界处填 <code>other</code>
<code>tl.store(ptr, val, mask)</code>	带掩码的块级存储
<code>tl.where(cond, a, b)</code>	逐元素条件选择（分段函数核心）
<code>tl.sum / tl.max / tl.min</code>	块内归约（归约算子核心）
<code>tl.atomic_add(ptr, val, mask)</code>	原子加（反向传播散射核心）
<code>tl.static_range</code>	编译期展开循环（核窗口迭代）
<code>@triton.jit</code>	即时编译装饰器
<code>@triton.autotune</code>	自动调优, 按 <code>key</code> 实测选配置
<code>@triton.heuristics</code>	启发式配置, 编译期推导常量
<code>flag_gems.libentry</code>	顶层 kernel 注册入口（封装 <code>jit</code> + 缓存）
<code>flag_gems.pointwise_dynamic</code>	逐点算子自动代码生成

References

- [1] P. Tillet, H. T. Kung, D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computers,” *Proc. MAPL*, 2019.
- [2] 智源研究院, “FlagGems: Triton-based GPU Operator Library,” <https://github.com/flagos-ai/FlagGems>, 2025.
- [3] S. Yang et al., “FLA: Flash Linear Attention Framework,” <https://github.com/sustcsonglin/flash-linear-attention>, 2024.
- [4] S. Yang et al., “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” *Proc. NeurIPS*, 2024.
- [5] T. Dao, D. Y. Fu, S. Ermon et al., “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *Proc. ICLR*, 2024.
- [6] NVIDIA Corporation, “CUDA C++ Programming Guide,” version 12.x, 2024.
- [7] C. Lattner et al., “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation,” *Proc. CGO*, 2021.
- [8] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” *Proc. ICML*, 2006.
- [9] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [10] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Proc. NeurIPS*, 2019.